# What is Bun?

Bun is an all-in-one toolkit for JavaScript and TypeScript apps. It ships as a single executable called bun.

At its core is the *Bun runtime*, a fast JavaScript runtime designed as **a drop-in replacement for Node.js**. It's written in Zig and powered by JavaScriptCore under the hood, dramatically reducing startup times and memory usage.

```
$ bun run index.tsx  # TS and JSX supported out of the box
```

The bun command-line tool also implements a test runner, script runner, and Node.js-compatible package manager, all significantly faster than existing tools and usable in existing Node.js projects with little to no changes necessary.

```
$ bun run start             # run the `start` script
$ bun install <pkg>         # install a package
$ bun build ./index.tsx     # bundle a project for browsers
$ bun test                  # run tests
$ bunx cowsay 'Hello, world!'   # execute a package
```

## What is a runtime?

JavaScript (or, more formally, ECMAScript) is just a *specification* for a programming language. Anyone can write a JavaScript *engine* that ingests a valid JavaScript program and executes it. The two most popular engines in use today are V8 (developed by Google) and JavaScriptCore (developed by Apple). Both are open source.

But most JavaScript programs don't run in a vacuum. They need a way to access the outside world to perform useful tasks. This is where *runtimes* come in. They implement additional APIs that are then made available to the JavaScript programs they execute.

### Browsers

Notably, browsers ship with JavaScript runtimes that implement a set of Web-specific APIs that are exposed via the global window object. Any JavaScript code executed by the browser can use these APIs to implement interactive or dynamic behavior in the context of the current webpage.

### Node.js

Similarly, Node.js is a JavaScript runtime that can be used in non-browser environments, like servers. JavaScript programs executed by Node.js have access to a set of Node.js-specific globals like Buffer, process, and __dirname in addition to built-in modules for performing OS-level tasks like reading/writing files (node:fs) and networking (node:net, node:http). Node.js also implements a CommonJS-based module system and resolution algorithm that pre-dates JavaScript's native module system.

Bun is designed as a faster, leaner, more modern replacement for Node.js.

## Design goals

Bun is designed from the ground-up with today's JavaScript ecosystem in mind.

- **Speed**. Bun processes start 4x faster than Node.js currently (try it yourself!)
- **TypeScript & JSX support**. You can directly execute .jsx, .ts, and .tsx files; Bun's transpiler converts these to vanilla JavaScript before execution.
- **ESM & CommonJS compatibility**. The world is moving towards ES modules (ESM), but millions of packages on npm still require CommonJS. Bun recommends ES modules, but supports CommonJS.
- **Web-standard APIs**. Bun implements standard Web APIs like fetch, WebSocket, and ReadableStream. Bun is powered by the JavaScriptCore engine, which is developed by Apple for Safari, so some APIs like Headers and URL directly use Safari's implementation.
- **Node.js compatibility**. In addition to supporting Node-style module resolution, Bun aims for full compatibility with built-in Node.js globals (process, Buffer) and modules (path, fs, http, etc.) *This is an ongoing effort that is not complete.* Refer to the compatibility page for the current status.

Bun is more than a runtime. The long-term goal is to be a cohesive, infrastructural toolkit for building apps with JavaScript/TypeScript, including a package manager, transpiler, bundler, script runner, test runner, and more.